

TripLink_client

Создано системой Doxygen 1.9.8

1	Алфавитный указатель пространств имен	1
1.1	Пространства имен	1
2	Иерархический список классов	3
2.1	Иерархия классов	3
3	Алфавитный указатель классов	5
3.1	Классы	5
4	Список файлов	7
4.1	Файлы	7
5	Пространства имен	9
5.1	Пространство имен Ui	9
6	Классы	11
6.1	Класс CarWindow	11
6.1.1	Подробное описание	12
6.1.2	Конструктор(ы)	12
6.1.2.1	CarWindow()	12
6.1.2.2	~CarWindow()	12
6.1.3	Методы	13
6.1.3.1	displayTripInfo	13
6.1.3.2	goToFinishWindow	13
6.1.3.3	on_toolButton_then0_clicked	13
6.1.3.4	on_toolButton_then1_clicked	13
6.1.3.5	returnToPreviousWindow	13
6.1.3.6	setTripId()	13
6.2	Класс CompanionWindow	14
6.2.1	Подробное описание	15
6.2.2	Конструктор(ы)	15
6.2.2.1	CompanionWindow()	15
6.2.2.2	~CompanionWindow()	16
6.2.3	Методы	16
6.2.3.1	getAvailableTrips()	16
6.2.3.2	goToCarWindow	16
6.2.3.3	goToDriverCompanionWindow	16
6.2.3.4	handleBookTripResponse	16
6.2.3.5	handleFindTripResponse	17
6.2.3.6	on_listWidget_info_itemClicked	17
6.2.3.7	on_toolButton_then0_clicked	17
6.2.3.8	on_toolButton_then1_clicked	17
6.2.3.9	returnToPreviousWindow	17
6.2.3.10	tripNotFound	18
6.3	Класс DriverCompanionWindow	18

6.3.1	Подробное описание	19
6.3.2	Конструктор(ы)	19
6.3.2.1	DriverCompanionWindow()	19
6.3.2.2	~DriverCompanionWindow()	19
6.3.3	Методы	19
6.3.3.1	goToCompanionWindow	19
6.3.3.2	goToDriverWindow	20
6.3.3.3	goToFeedbackWindow	20
6.3.3.4	returnToPreviousWindow	20
6.4	Класс DriverWindow	20
6.4.1	Подробное описание	21
6.4.2	Конструктор(ы)	21
6.4.2.1	DriverWindow()	21
6.4.2.2	~DriverWindow()	22
6.4.3	Методы	22
6.4.3.1	goToFinishWindow	22
6.4.3.2	returnToPreviousWindow	22
6.5	Класс FinishWindow	22
6.5.1	Подробное описание	23
6.5.2	Конструктор(ы)	23
6.5.2.1	FinishWindow()	23
6.5.2.2	~FinishWindow()	24
6.5.3	Методы	24
6.5.3.1	returnToMainWindow	24
6.6	Класс LoginWindow	24
6.6.1	Подробное описание	25
6.6.2	Конструктор(ы)	25
6.6.2.1	LoginWindow()	25
6.6.2.2	~LoginWindow()	25
6.6.3	Методы	25
6.6.3.1	goToDriverCompanionWindow	25
6.6.3.2	returnToMainWindow	26
6.7	Класс MainWindow	26
6.7.1	Подробное описание	27
6.7.2	Конструктор(ы)	27
6.7.2.1	MainWindow()	27
6.7.2.2	~MainWindow()	27
6.7.3	Методы	28
6.7.3.1	loginButtonClicked	28
6.7.3.2	registrationButtonClicked	28
6.8	Класс ManagerForm	28
6.8.1	Подробное описание	28
6.9	Класс NetworkClient	29

6.9.1	Подробное описание	30
6.9.2	Конструктор(ы)	30
6.9.2.1	NetworkClient()	30
6.9.3	Методы	31
6.9.3.1	_readyRead	31
6.9.3.2	authFailed	31
6.9.3.3	authSuccess	31
6.9.3.4	bookTripResponse	31
6.9.3.5	connectedToServer	31
6.9.3.6	connectionStatusChanged	31
6.9.3.7	connectToServer()	32
6.9.3.8	disconnectedFromServer	32
6.9.3.9	error	32
6.9.3.10	getInstance()	32
6.9.3.11	getLogin()	32
6.9.3.12	operator=()	33
6.9.3.13	readyRead	33
6.9.3.14	regFailed	33
6.9.3.15	regSuccess	33
6.9.3.16	sendMessage()	33
6.9.3.17	socketError	33
6.10	Класс RegistrationWindow	34
6.10.1	Подробное описание	35
6.10.2	Конструктор(ы)	35
6.10.2.1	RegistrationWindow()	35
6.10.2.2	~RegistrationWindow()	35
6.10.3	Методы	35
6.10.3.1	goToDriverCompanionWindow	35
6.10.3.2	returnToMainWindow	35
6.11	Класс Trips	36
6.11.1	Подробное описание	37
6.11.2	Конструктор(ы)	37
6.11.2.1	Trips()	37
6.11.2.2	~Trips()	37
6.11.3	Методы	37
6.11.3.1	finished	37
6.11.3.2	goToDriverCompanionWindow	37
7	Файлы	39
7.1	Файл client/carwindow.cpp	39
7.1.1	Подробное описание	39
7.2	Файл client/carwindow.h	39
7.2.1	Подробное описание	40

7.3 carwindow.h	41
7.4 Файл client/companionwindow.cpp	41
7.4.1 Подробное описание	41
7.5 Файл client/companionwindow.h	42
7.5.1 Подробное описание	42
7.6 companionwindow.h	43
7.7 Файл client/drivercompanionwindow.cpp	43
7.7.1 Подробное описание	44
7.8 Файл client/drivercompanionwindow.h	44
7.8.1 Подробное описание	45
7.9 drivercompanionwindow.h	45
7.10 Файл client/driverwindow.cpp	46
7.10.1 Подробное описание	46
7.11 Файл client/driverwindow.h	47
7.11.1 Подробное описание	48
7.12 driverwindow.h	48
7.13 Файл client/feedback.cpp	48
7.13.1 Подробное описание	49
7.14 Файл client/feedback.h	49
7.14.1 Подробное описание	50
7.15 feedback.h	50
7.16 Файл client/finishwindow.cpp	51
7.16.1 Подробное описание	51
7.17 Файл client/finishwindow.h	51
7.17.1 Подробное описание	52
7.18 finishwindow.h	53
7.19 Файл client/loginwindow.cpp	53
7.19.1 Подробное описание	53
7.20 Файл client/loginwindow.h	54
7.20.1 Подробное описание	54
7.21 loginwindow.h	55
7.22 Файл client/main.cpp	55
7.22.1 Подробное описание	56
7.22.2 Функции	56
7.22.2.1 main()	56
7.23 Файл client/mainwindow.cpp	56
7.23.1 Подробное описание	56
7.24 Файл client/mainwindow.h	57
7.24.1 Подробное описание	57
7.25 mainwindow.h	58
7.26 Файл client/managerform.cpp	58
7.26.1 Подробное описание	58
7.27 Файл client/managerform.h	59

7.27.1 Подробное описание	59
7.28 managerform.h	59
7.29 Файл client/networkclient.cpp	60
7.29.1 Подробное описание	61
7.30 Файл client/networkclient.h	61
7.30.1 Подробное описание	62
7.31 networkclient.h	62
7.32 Файл client/registrationwindow.cpp	63
7.32.1 Подробное описание	63
7.33 Файл client/registrationwindow.h	63
7.33.1 Подробное описание	64
7.34 registrationwindow.h	64
Предметный указатель	67

Глава 1

Алфавитный указатель пространств имен

1.1 Пространства имен

Полный список пространств имен.

Ui	9
--------------	---

Глава 2

Иерархический список классов

2.1 Иерархия классов

Иерархия классов.

ManagerForm	28
QDialog	
CarWindow	11
CompanionWindow	14
DriverCompanionWindow	18
DriverWindow	20
FinishWindow	22
LoginWindow	24
MainWindow	26
RegistrationWindow	34
Tips	36
QObject	
NetworkClient	29

Глава 3

Алфавитный указатель классов

3.1 Классы

Классы с их кратким описанием.

CarWindow	Класс CarWindow представляет окно с информацией о поездках. Окно отображает информацию о доступных поездках и предоставляет возможность выбрать поездку	11
CompanionWindow	Класс CompanionWindow представляет окно для выбора и бронирования поездок пассажиром. Окно позволяет искать доступные поездки, выбирать их и бронировать	14
DriverCompanionWindow	Класс DriverCompanionWindow представляет окно для выбора роли водителя, пассажира, или оставления отзыва. Окно предоставляет возможность возврата на предыдущий экран или перехода к различным экранам	18
DriverWindow	Класс для представления окна водителя	20
FinishWindow	Класс окна завершения	22
LoginWindow	Класс окна авторизации	24
MainWindow	Класс главного окна приложения	26
ManagerForm	Класс, управляющий окнами приложения	28
NetworkClient	Класс для работы с сетевым клиентом	29
RegistrationWindow	Класс окна регистрации пользователя	34
Trips	Класс окна обратной связи	36

Глава 4

Список файлов

4.1 Файлы

Полный список файлов.

client/ carwindow.cpp	Файл реализации класса CarWindow	39
client/ carwindow.h	Заголовочный файл класса CarWindow , реализующего окно с информацией о доступных поездках	39
client/ companionwindow.cpp	Файл реализации класса CompanionWindow	41
client/ companionwindow.h	Заголовочный файл класса CompanionWindow , реализующего окно для выбора и бронирования поездок пассажиром	42
client/ drivercompanionwindow.cpp	Файл реализации класса DriverCompanionWindow	43
client/ drivercompanionwindow.h	Заголовочный файл класса DriverCompanionWindow , реализующего окно выбора роли (пассажир, водитель)	44
client/ driverwindow.cpp	Файл реализации класса DriverWindow	46
client/ driverwindow.h	Заголовочный файл класса DriverWindow , реализующего окно для водителя	47
client/ feedback.cpp	Файл реализации класса Tpips	48
client/ feedback.h	Заголовочный файл класса Tpips , реализующего возможность оставления отзывов и рейтинга поездок	49
client/ finishwindow.cpp	Файл реализации класса FinishWindow	51
client/ finishwindow.h	Заголовочный файл класса FinishWindow , реализующего окно завершения	51
client/ loginwindow.cpp	Файл реализации класса LoginWindow	53
client/ loginwindow.h	Заголовочный файл класса LoginWindow , реализующего окно авторизации	54
client/ main.cpp	Главный файл приложения	55
client/ mainwindow.cpp	Файл реализации класса MainWindow	56

client/ mainwindow.h	
Заголовочный файл класса MainWindow , реализующего главное окно приложения	57
client/ managerform.cpp	
Файл реализации класса ManagerForm	58
client/ managerform.h	
Заголовочный файл класса ManagerForm для управления окнами приложения . .	59
client/ networkclient.cpp	
Файл реализации класса RegistrationWindow	60
client/ networkclient.h	
Заголовочный файл класса RegistrationWindow для работы с сетевым клиентом .	61
client/ registrationwindow.cpp	
Файл реализации класса RegistrationWindow	63
client/ registrationwindow.h	
Заголовочный файл класса RegistrationWindow для реализации окна регистрации	63

Глава 5

Пространства имен

5.1 Пространство имен U_i

Глава 6

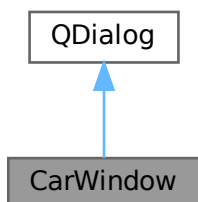
Классы

6.1 Класс CarWindow

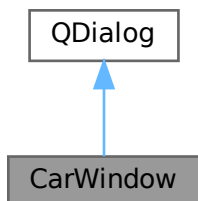
Класс `CarWindow` представляет окно с информацией о поездках. Окно отображает информацию о доступных поездках и предоставляет возможность выбрать поездку.

```
#include <carwindow.h>
```

Граф наследования: `CarWindow`:



Граф связей класса `CarWindow`:



Открытые слоты

- void `on_toolButton_then0_clicked` ()
Обработчик события нажатия на кнопку "then0".
- void `on_toolButton_then1_clicked` ()
Обработчик события нажатия на кнопку "then1".
- void `displayTripInfo` (const QVariantMap &tripInfo)
Слот для отображения информации о поездке.

Сигналы

- void `returnToPreviousWindow` ()
Сигнал для возврата на предыдущий экран.
- void `goToFinishWindow` ()
Сигнал для перехода на экран завершения.

Открытые члены

- `CarWindow` (QWidget *parent=nullptr)
Конструктор `CarWindow`.
- `~CarWindow` ()
Деструктор `CarWindow`.
- void `setTripId` (int tripId)
Установить ID поездки.

6.1.1 Подробное описание

Класс `CarWindow` представляет окно с информацией о поездках. Окно отображает информацию о доступных поездках и предоставляет возможность выбрать поездку.

6.1.2 Конструктор(ы)

6.1.2.1 `CarWindow()`

```
CarWindow::CarWindow (
    QWidget * parent = nullptr ) [explicit]
```

Конструктор `CarWindow`.

Аргументы

parent	Указатель на родительский виджет.
--------	-----------------------------------

6.1.2.2 `~CarWindow()`

```
CarWindow::~~CarWindow ( )
```

Деструктор [CarWindow](#).

6.1.3 Методы

6.1.3.1 displayTripInfo

```
void CarWindow::displayTripInfo (
    const QMap & tripInfo ) [slot]
```

Слот для отображения информации о поездке.

Аргументы

tripInfo	Информация о поездке.
----------	-----------------------

6.1.3.2 goToFinishWindow

```
void CarWindow::goToFinishWindow ( ) [signal]
```

Сигнал для перехода на экран завершения.

6.1.3.3 on_toolButton_then0_clicked

```
void CarWindow::on_toolButton_then0_clicked ( ) [slot]
```

Обработчик события нажатия на кнопку "then0".

6.1.3.4 on_toolButton_then1_clicked

```
void CarWindow::on_toolButton_then1_clicked ( ) [slot]
```

Обработчик события нажатия на кнопку "then1".

6.1.3.5 returnToPreviousWindow

```
void CarWindow::returnToPreviousWindow ( ) [signal]
```

Сигнал для возврата на предыдущий экран.

6.1.3.6 setTripId()

```
void CarWindow::setTripId (
    int tripId )
```

Установить ID поездки.

Аргументы

tripId	ID поездки.
--------	-------------

Объявления и описания членов классов находятся в файлах:

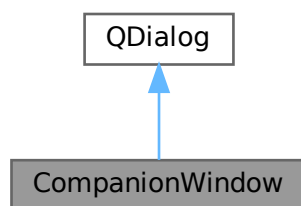
- [client/carwindow.h](#)
- [client/carwindow.cpp](#)

6.2 Класс CompanionWindow

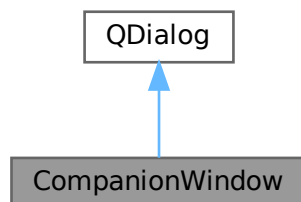
Класс [CompanionWindow](#) представляет окно для выбора и бронирования поездок пассажиром. Окно позволяет искать доступные поездки, выбирать их и бронировать.

```
#include <companionwindow.h>
```

Граф наследования:CompanionWindow:



Граф связей класса CompanionWindow:



Открытые слоты

- void [on_toolButton_then0_clicked](#) ()
Обработчик события нажатия на кнопку "then0".
- void [on_toolButton_then1_clicked](#) ()
Обработчик события нажатия на кнопку "then1".
- void [handleFindTripResponse](#) (const QString &message)
Слот для обработки ответа на запрос о поиске поездки.
- void [handleBookTripResponse](#) (const QString &message)
Слот для обработки ответа на запрос о бронировании поездки.
- void [on_listWidget_info_itemClicked](#) (QListWidgetItem *item)
Обработчик события выбора поездки из списка.

Сигналы

- void [returnToPreviousWindow](#) ()
Сигнал для возврата на предыдущий экран.
- void [goToCarWindow](#) (int tripId, QVariantMap tripInfo)
Сигнал для перехода на экран с информацией о выбранной поездке.
- void [tripNotFound](#) ()
Сигнал, если поездка не найдена.
- void [goToDriverCompanionWindow](#) ()
Сигнал для возврата на экран [DriverCompanionWindow](#).

Открытые члены

- [CompanionWindow](#) (QWidget *parent=nullptr)
Конструктор [CompanionWindow](#).
- [~CompanionWindow](#) ()
Деструктор [CompanionWindow](#).
- QVector< QVariantMap > [getAvailableTrips](#) () const
Получить доступные поездки.

6.2.1 Подробное описание

Класс [CompanionWindow](#) представляет окно для выбора и бронирования поездок пассажиром. Окно позволяет искать доступные поездки, выбирать их и бронировать.

6.2.2 Конструктор(ы)

6.2.2.1 CompanionWindow()

```
CompanionWindow::CompanionWindow (  
    QWidget * parent = nullptr ) [explicit]
```

Конструктор [CompanionWindow](#).

Аргументы

parent	Указатель на родительский виджет.
--------	-----------------------------------

6.2.2.2 ~CompanionWindow()

CompanionWindow::~~CompanionWindow ()

Деструктор [CompanionWindow](#).

6.2.3 Методы

6.2.3.1 getAvailableTrips()

QVector< QVariantMap > CompanionWindow::getAvailableTrips () const

Получить доступные поездки.

Возвращает

Вектор с данными доступных поездок.

6.2.3.2 goToCarWindow

```
void CompanionWindow::goToCarWindow (
    int tripId,
    QVariantMap tripInfo ) [signal]
```

Сигнал для перехода на экран с информацией о выбранной поездке.

Аргументы

tripId	ID выбранной поездки.
tripInfo	Информация о поездке.

6.2.3.3 goToDriverCompanionWindow

```
void CompanionWindow::goToDriverCompanionWindow ( ) [signal]
```

Сигнал для возврата на экран [DriverCompanionWindow](#).

6.2.3.4 handleBookTripResponse

```
void CompanionWindow::handleBookTripResponse (
    const QString & message ) [slot]
```

Слот для обработки ответа на запрос о бронировании поездки.

Аргументы

message	Ответ от сервера.
---------	-------------------

6.2.3.5 handleFindTripResponse

```
void CompanionWindow::handleFindTripResponse (
    const QString & message ) [slot]
```

Слот для обработки ответа на запрос о поиске поездки.

Аргументы

message	Ответ от сервера.
---------	-------------------

6.2.3.6 on_listWidget_info_itemClicked

```
void CompanionWindow::on_listWidget_info_itemClicked (
    QListWidgetItem * item ) [slot]
```

Обработчик события выбора поездки из списка.

Аргументы

item	Элемент списка, соответствующий выбранной поездке.
------	--

6.2.3.7 on_toolButton_then0_clicked

```
void CompanionWindow::on_toolButton_then0_clicked ( ) [slot]
```

Обработчик события нажатия на кнопку "then0".

6.2.3.8 on_toolButton_then1_clicked

```
void CompanionWindow::on_toolButton_then1_clicked ( ) [slot]
```

Обработчик события нажатия на кнопку "then1".

6.2.3.9 returnToPreviousWindow

```
void CompanionWindow::returnToPreviousWindow ( ) [signal]
```

Сигнал для возврата на предыдущий экран.

6.2.3.10 tripNotFound

```
void CompanionWindow::tripNotFound ( ) [signal]
```

Сигнал, если поездка не найдена.

Объявления и описания членов классов находятся в файлах:

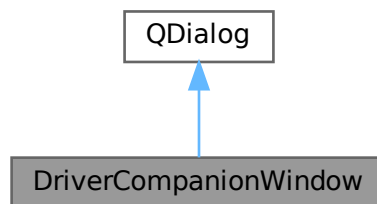
- [client/companionwindow.h](#)
- [client/companionwindow.cpp](#)

6.3 Класс DriverCompanionWindow

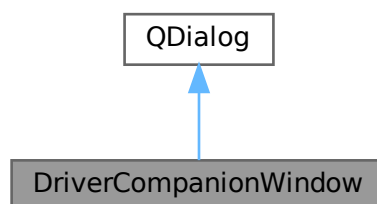
Класс [DriverCompanionWindow](#) представляет окно для выбора роли водителя, пассажира, или оставления отзыва. Окно предоставляет возможность возврата на предыдущий экран или перехода к различным экранам.

```
#include <drivercompanionwindow.h>
```

Граф наследования:DriverCompanionWindow:



Граф связей класса DriverCompanionWindow:



Сигналы

- void [returnToPreviousWindow](#) ()
Сигнал для возврата на предыдущий экран.
- void [goToCompanionWindow](#) ()
Сигнал для перехода на экран с компаньоном.
- void [goToDriverWindow](#) ()
Сигнал для перехода на экран водителя.
- void [goToFeedbackWindow](#) ()
Сигнал для перехода на экран отзывов.

Открытые члены

- [DriverCompanionWindow](#) (QWidget *parent=nullptr)
Конструктор [DriverCompanionWindow](#).
- [~DriverCompanionWindow](#) ()
Деструктор [DriverCompanionWindow](#).

6.3.1 Подробное описание

Класс [DriverCompanionWindow](#) представляет окно для выбора роли водителя, пассажира, или оставления отзыва. Окно предоставляет возможность возврата на предыдущий экран или перехода к различным экранам.

6.3.2 Конструктор(ы)

6.3.2.1 DriverCompanionWindow()

```
DriverCompanionWindow::DriverCompanionWindow (
    QWidget * parent = nullptr ) [explicit]
```

Конструктор [DriverCompanionWindow](#).

Аргументы

parent	Указатель на родительский виджет.
--------	-----------------------------------

6.3.2.2 ~DriverCompanionWindow()

```
DriverCompanionWindow::~~DriverCompanionWindow ( )
```

Деструктор [DriverCompanionWindow](#).

6.3.3 Методы

6.3.3.1 goToCompanionWindow

```
void DriverCompanionWindow::goToCompanionWindow ( ) [signal]
```

Сигнал для перехода на экран с компаньоном.

6.3.3.2 goToDriverWindow

```
void DriverCompanionWindow::goToDriverWindow ( ) [signal]
```

Сигнал для перехода на экран водителя.

6.3.3.3 goToFeedbackWindow

```
void DriverCompanionWindow::goToFeedbackWindow ( ) [signal]
```

Сигнал для перехода на экран отзывов.

6.3.3.4 returnToPreviousWindow

```
void DriverCompanionWindow::returnToPreviousWindow ( ) [signal]
```

Сигнал для возврата на предыдущий экран.

Объявления и описания членов классов находятся в файлах:

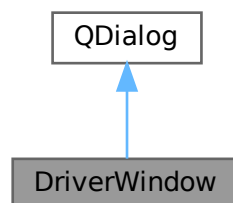
- [client/drivercompanionwindow.h](#)
- [client/drivercompanionwindow.cpp](#)

6.4 Класс DriverWindow

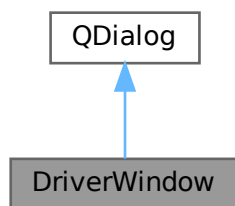
Класс для представления окна водителя.

```
#include <driverwindow.h>
```

Граф наследования:DriverWindow:



Граф связей класса DriverWindow:



Сигналы

- void `returnToPreviousWindow` ()
Сигнал для возвращения на предыдущее окно.
- void `goToFinishWindow` ()
Сигнал для перехода на окно завершения.

Открытые члены

- `DriverWindow` (QWidget *parent=nullptr)
Конструктор класса `DriverWindow`.
- `~DriverWindow` ()
Деструктор класса `DriverWindow`.

6.4.1 Подробное описание

Класс для представления окна водителя.

Этот класс реализует окно водителя, позволяя ему создавать поездки и отправлять данные на сервер.

6.4.2 Конструктор(ы)

6.4.2.1 DriverWindow()

```
DriverWindow::DriverWindow (  
    QWidget * parent = nullptr ) [explicit]
```

Конструктор класса `DriverWindow`.

Инициализирует окно водителя.

Аргументы

parent	Указатель на родительский виджет (по умолчанию nullptr).
--------	--

6.4.2.2 ~DriverWindow()

DriverWindow::~DriverWindow ()

Деструктор класса [DriverWindow](#).

Удаляет все ресурсы, связанные с окном водителя.

6.4.3 Методы

6.4.3.1 goToFinishWindow

void DriverWindow::goToFinishWindow () [signal]

Сигнал для перехода на окно завершения.

6.4.3.2 returnToPreviousWindow

void DriverWindow::returnToPreviousWindow () [signal]

Сигнал для возвращения на предыдущее окно.

Объявления и описания членов классов находятся в файлах:

- [client/driverwindow.h](#)
- [client/driverwindow.cpp](#)

6.5 Класс FinishWindow

Класс окна завершения.

```
#include <finishwindow.h>
```

Граф наследования:FinishWindow:



Граф связей класса FinishWindow:



Сигналы

- void `returnToMainWindow` ()
Сигнал для возврата на главное окно.

Открытые члены

- `FinishWindow` (QWidget *parent=nullptr)
Конструктор класса `FinishWindow`.
- `~FinishWindow` ()
Деструктор класса `FinishWindow`.

6.5.1 Подробное описание

Класс окна завершения.

Этот класс реализует окно завершения, которое позволяет пользователю вернуться на главный экран.

6.5.2 Конструктор(ы)

6.5.2.1 FinishWindow()

```
FinishWindow::FinishWindow (  
    QWidget * parent = nullptr ) [explicit]
```

Конструктор класса `FinishWindow`.

Инициализирует интерфейс окна завершения.

Аргументы

parent	Родительский элемент (по умолчанию nullptr).
--------	--

6.5.2.2 ~FinishWindow()

FinishWindow::~~FinishWindow ()

Деструктор класса [FinishWindow](#).

Освобождает ресурсы, связанные с интерфейсом окна завершения.

6.5.3 Методы

6.5.3.1 returnToMainWindow

void FinishWindow::returnToMainWindow () [signal]

Сигнал для возврата на главное окно.

Объявления и описания членов классов находятся в файлах:

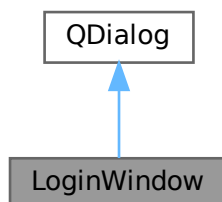
- [client/finishwindow.h](#)
- [client/finishwindow.cpp](#)

6.6 Класс LoginWindow

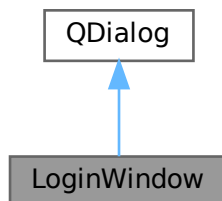
Класс окна авторизации.

```
#include <loginwindow.h>
```

Граф наследования:LoginWindow:



Граф связей класса LoginWindow:



Сигналы

- void `returnToMainWindow` ()
Сигнал для возврата в главное окно.
- void `goToDriverCompanionWindow` ()
Сигнал для перехода к окну "DriverCompanionWindow".

Открытые члены

- `LoginWindow` (QWidget *parent=nullptr)
Конструктор класса `LoginWindow`.
- `~LoginWindow` ()
Деструктор класса `LoginWindow`.

6.6.1 Подробное описание

Класс окна авторизации.

Этот класс реализует окно авторизации, которое позволяет пользователю вводить логин и пароль. После успешной или неудачной авторизации генерируются соответствующие сигналы.

6.6.2 Конструктор(ы)

6.6.2.1 LoginWindow()

```
LoginWindow::LoginWindow (
    QWidget * parent = nullptr ) [explicit]
```

Конструктор класса `LoginWindow`.

Инициализирует интерфейс и подключает сигналы для обработки событий авторизации.

Аргументы

parent	Родительский элемент (по умолчанию nullptr).
--------	--

6.6.2.2 ~LoginWindow()

```
LoginWindow::~LoginWindow ( )
```

Деструктор класса `LoginWindow`.

Освобождает ресурсы, связанные с интерфейсом окна.

6.6.3 Методы

6.6.3.1 goToDriverCompanionWindow

```
void LoginWindow::goToDriverCompanionWindow ( ) [signal]
```

Сигнал для перехода к окну "DriverCompanionWindow".

6.6.3.2 returnToMainWindow

```
void LoginWindow::returnToMainWindow ( ) [signal]
```

Сигнал для возврата в главное окно.

Объявления и описания членов классов находятся в файлах:

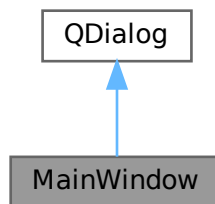
- [client/loginwindow.h](#)
- [client/loginwindow.cpp](#)

6.7 Класс MainWindow

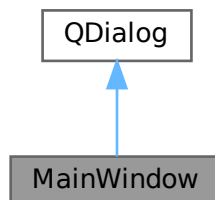
Класс главного окна приложения.

```
#include <mainwindow.h>
```

Граф наследования:MainWindow:



Граф связей класса MainWindow:



Сигналы

- void `loginButtonClicked` ()
Сигнал, генерируемый при нажатии кнопки входа.
- void `registrationButtonClicked` ()
Сигнал, генерируемый при нажатии кнопки регистрации.

Открытые члены

- `MainWindow` (QWidget *parent=nullptr)
Конструктор класса `MainWindow`.
- `~MainWindow` ()
Деструктор класса `MainWindow`.

6.7.1 Подробное описание

Класс главного окна приложения.

Этот класс реализует главное окно приложения с двумя кнопками: для входа и для регистрации. При нажатии на соответствующие кнопки генерируются сигналы для перехода к окнам авторизации или регистрации. Также устанавливается соединение с сервером при инициализации окна.

6.7.2 Конструктор(ы)

6.7.2.1 MainWindow()

```
MainWindow::MainWindow (
    QWidget * parent = nullptr ) [explicit]
```

Конструктор класса `MainWindow`.

Создает главное окно приложения, инициализирует элементы интерфейса и устанавливает соединение с сервером.

Аргументы

parent	Родительский элемент (по умолчанию nullptr).
--------	--

6.7.2.2 ~MainWindow()

```
MainWindow::~MainWindow ( )
```

Деструктор класса `MainWindow`.

Освобождает ресурсы, связанные с интерфейсом главного окна.

6.7.3 Методы

6.7.3.1 loginButtonClicked

```
void MainWindow::loginButtonClicked ( ) [signal]
```

Сигнал, генерируемый при нажатии кнопки входа.

6.7.3.2 registrationButtonClicked

```
void MainWindow::registrationButtonClicked ( ) [signal]
```

Сигнал, генерируемый при нажатии кнопки регистрации.

Объявления и описания членов классов находятся в файлах:

- [client/mainwindow.h](#)
- [client/mainwindow.cpp](#)

6.8 Класс ManagerForm

Класс, управляющий окнами приложения.

```
#include <managerform.h>
```

6.8.1 Подробное описание

Класс, управляющий окнами приложения.

Этот класс реализует главный менеджер, который управляет навигацией между различными окнами в приложении. Он организует взаимодействие с окнами, такими как: вход, регистрация, водитель, компаньон, и т.д.

Объявления и описания членов класса находятся в файле:

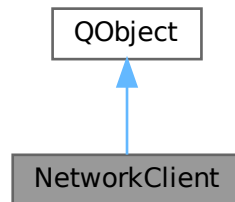
- [client/managerform.h](#)

6.9 Класс NetworkClient

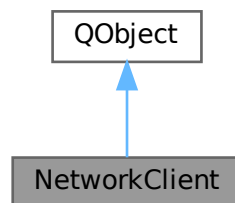
Класс для работы с сетевым клиентом.

```
#include <networkclient.h>
```

Граф наследования:NetworkClient:



Граф связей класса NetworkClient:



Открытые слоты

- void `connectedToServer` ()
Слот, вызываемый при успешном подключении к серверу.
- void `disconnectedFromServer` ()
Слот, вызываемый при отключении от сервера.
- void `socketError` (QAbstractSocket::SocketError socketError)
Слот для обработки ошибки сокета.
- void `_readyRead` ()
Слот для обработки входящих данных от сервера.

Сигналы

- void `connectionStatusChanged` (bool connected)
Сигнал о статусе соединения.
- void `error` (const QString &message)
Сигнал об ошибке.
- void `authSuccess` ()
Сигнал об успешной авторизации.
- void `authFailed` ()
Сигнал о неудачной авторизации.
- void `regSuccess` ()
Сигнал об успешной регистрации.
- void `regFailed` ()
Сигнал о неудачной регистрации.
- void `readyRead` (const QString &message)
Сигнал для получения сообщения от сервера.
- void `bookTripResponse` (const QString &response)
Сигнал для получения ответа на запрос о бронировании поездки.

Открытые члены

- `NetworkClient` (`NetworkClient` const &)=delete
- void `operator=` (`NetworkClient` const &)=delete
- void `connectToServer` (const QString &host, quint16 port)
Метод для подключения к серверу.
- void `sendMessage` (const QString &message)
Метод для отправки сообщения на сервер.
- QString `getLogin` () const
Метод для получения логина текущего пользователя.

Открытые статические члены

- static `NetworkClient` & `getInstance` ()
Метод для получения экземпляра синглтона.

6.9.1 Подробное описание

Класс для работы с сетевым клиентом.

Этот класс реализует взаимодействие с сервером через TCP-сокет. Он реализует синглтон, обеспечивая наличие только одного экземпляра класса в процессе. Основные функции: подключение к серверу, отправка сообщений, обработка ошибок, получение данных.

6.9.2 Конструктор(ы)

6.9.2.1 NetworkClient()

```
NetworkClient::NetworkClient (
    NetworkClient const & ) [delete]
```

6.9.3 Методы

6.9.3.1 _readyRead

```
void NetworkClient::_readyRead ( ) [slot]
```

Слот для обработки входящих данных от сервера.

6.9.3.2 authFailed

```
void NetworkClient::authFailed ( ) [signal]
```

Сигнал о неудачной авторизации.

6.9.3.3 authSuccess

```
void NetworkClient::authSuccess ( ) [signal]
```

Сигнал об успешной авторизации.

6.9.3.4 bookTripResponse

```
void NetworkClient::bookTripResponse (
    const QString & response ) [signal]
```

Сигнал для получения ответа на запрос о бронировании поездки.

Аргументы

response	Ответ сервера.
----------	----------------

6.9.3.5 connectedToServer

```
void NetworkClient::connectedToServer ( ) [slot]
```

Слот, вызываемый при успешном подключении к серверу.

6.9.3.6 connectionStatusChanged

```
void NetworkClient::connectionStatusChanged (
    bool connected ) [signal]
```

Сигнал о статусе соединения.

Аргументы

connected	Статус подключения (true - подключено, false - отключено).
-----------	--

6.9.3.7 connectToServer()

```
void NetworkClient::connectToServer (
    const QString & host,
    quint16 port )
```

Метод для подключения к серверу.

Аргументы

host	Адрес хоста.
port	Порт для подключения.

6.9.3.8 disconnectedFromServer

```
void NetworkClient::disconnectedFromServer ( ) [slot]
```

Слот, вызываемый при отключении от сервера.

6.9.3.9 error

```
void NetworkClient::error (
    const QString & message ) [signal]
```

Сигнал об ошибке.

Аргументы

message	Сообщение об ошибке.
---------	----------------------

6.9.3.10 getInstance()

```
static NetworkClient & NetworkClient::getInstance ( ) [inline], [static]
```

Метод для получения экземпляра синглтона.

Возвращает

Экземпляр класса [NetworkClient](#).

6.9.3.11 getLogin()

```
QString NetworkClient::getLogin ( ) const [inline]
```

Метод для получения логина текущего пользователя.

Возвращает

Логин текущего пользователя.

6.9.3.12 operator=()

```
void NetworkClient::operator= (
    NetworkClient const & ) [delete]
```

6.9.3.13 readyRead

```
void NetworkClient::readyRead (
    const QString & message ) [signal]
```

Сигнал для получения сообщения от сервера.

Аргументы

message	Сообщение.
---------	------------

6.9.3.14 regFailed

```
void NetworkClient::regFailed ( ) [signal]
```

Сигнал о неудачной регистрации.

6.9.3.15 regSuccess

```
void NetworkClient::regSuccess ( ) [signal]
```

Сигнал об успешной регистрации.

6.9.3.16 sendMessage()

```
void NetworkClient::sendMessage (
    const QString & message )
```

Метод для отправки сообщения на сервер.

Аргументы

message	Сообщение для отправки.
---------	-------------------------

6.9.3.17 socketError

```
void NetworkClient::socketError (
    QAbstractSocket::SocketError socketError ) [slot]
```

Слот для обработки ошибки сокета.

Аргументы

socketError	Тип ошибки сокета.
-------------	--------------------

Объявления и описания членов классов находятся в файлах:

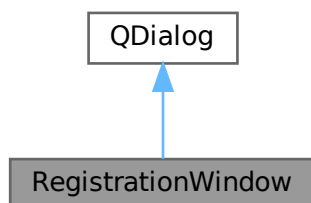
- client/[networkclient.h](#)
- client/[networkclient.cpp](#)

6.10 Класс RegistrationWindow

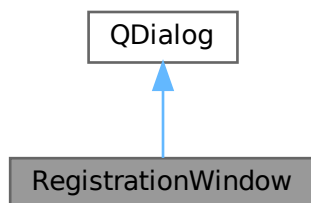
Класс окна регистрации пользователя.

```
#include <registrationwindow.h>
```

Граф наследования:RegistrationWindow:



Граф связей класса RegistrationWindow:



Сигналы

- void [returnToMainWindow](#) ()
Сигнал для возврата на главное окно.
- void [goToDriverCompanionWindow](#) ()
Сигнал для перехода в окно водителя-сопутника.

Открытые члены

- [RegistrationWindow](#) (QWidget *parent=nullptr)
Конструктор.
- [~RegistrationWindow](#) ()
Деструктор.

6.10.1 Подробное описание

Класс окна регистрации пользователя.

Этот класс реализует окно, которое позволяет пользователю зарегистрироваться в системе. Он включает обработку ввода логина, пароля и email, а также взаимодействует с сервером для выполнения регистрации.

6.10.2 Конструктор(ы)

6.10.2.1 RegistrationWindow()

```
RegistrationWindow::RegistrationWindow (
    QWidget * parent = nullptr ) [explicit]
```

Конструктор.

Аргументы

parent	Родительский элемент (по умолчанию nullptr).
--------	--

6.10.2.2 ~RegistrationWindow()

```
RegistrationWindow::~RegistrationWindow ( )
```

Деструктор.

6.10.3 Методы

6.10.3.1 goToDriverCompanionWindow

```
void RegistrationWindow::goToDriverCompanionWindow ( ) [signal]
```

Сигнал для перехода в окно водителя-сопутника.

6.10.3.2 returnToMainWindow

```
void RegistrationWindow::returnToMainWindow ( ) [signal]
```

Сигнал для возврата на главное окно.

Объявления и описания членов классов находятся в файлах:

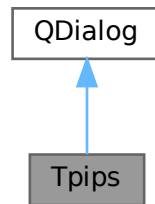
- [client/registrationwindow.h](#)
- [client/registrationwindow.cpp](#)

6.11 Класс Ttips

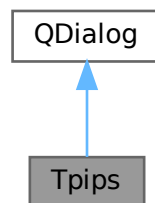
Класс окна обратной связи.

```
#include <feedback.h>
```

Граф наследования: Ttips:



Граф связей класса Ttips:



Сигналы

- void `finished ()`
Сигнал для завершения работы и возврата.
- void `goToDriverCompanionWindow ()`
Сигнал для перехода в окно "DriverCompanionWindow".

Открытые члены

- `Ttips (QWidget *parent=nullptr)`
Конструктор класса `Ttips`.
- `~Ttips ()`
Деструктор класса `Ttips`.

6.11.1 Подробное описание

Класс окна обратной связи.

Этот класс реализует окно для ввода отзывов и оценки, а также отправки этих данных на сервер.

6.11.2 Конструктор(ы)

6.11.2.1 Trips()

```
Trips::Trips (
    QWidget * parent = nullptr ) [explicit]
```

Конструктор класса [Trips](#).

Инициализирует интерфейс окна обратной связи и подключает сигналы для обработки действий пользователя.

Аргументы

parent	Родительский элемент (по умолчанию nullptr).
--------	--

6.11.2.2 ~Trips()

```
Trips::~Trips ( )
```

Деструктор класса [Trips](#).

Освобождает ресурсы, связанные с интерфейсом окна обратной связи.

6.11.3 Методы

6.11.3.1 finished

```
void Trips::finished ( ) [signal]
```

Сигнал для завершения работы и возврата.

6.11.3.2 goToDriverCompanionWindow

```
void Trips::goToDriverCompanionWindow ( ) [signal]
```

Сигнал для перехода в окно "DriverCompanionWindow".

Объявления и описания членов классов находятся в файлах:

- [client/feedback.h](#)
- [client/feedback.cpp](#)

Глава 7

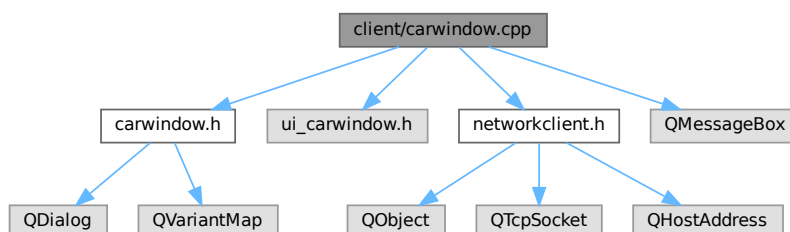
Файлы

7.1 Файл client/carwindow.cpp

Файл реализации класса [CarWindow](#).

```
#include "carwindow.h"  
#include "ui_carwindow.h"  
#include "networkclient.h"  
#include <QMessageBox>
```

Граф включаемых заголовочных файлов для carwindow.cpp:



7.1.1 Подробное описание

Файл реализации класса [CarWindow](#).

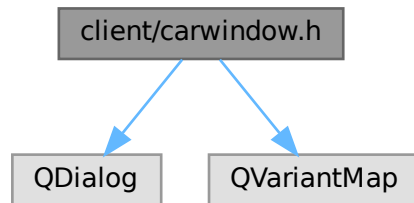
7.2 Файл client/carwindow.h

Заголовочный файл класса [CarWindow](#), реализующего окно с информацией о доступных поездках.

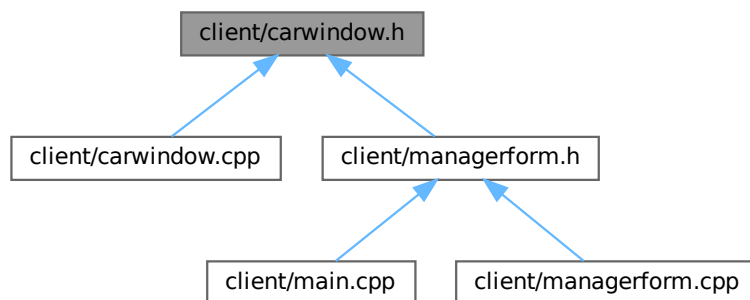
```
#include <QDialog>
```

```
#include <QVariantMap>
```

Граф включаемых заголовочных файлов для carwindow.h:



Граф файлов, в которые включается этот файл:



Классы

- class [CarWindow](#)

Класс [CarWindow](#) представляет окно с информацией о поездках. Окно отображает информацию о доступных поездках и предоставляет возможность выбрать поездку.

Пространства имен

- namespace [Ui](#)

7.2.1 Подробное описание

Заголовочный файл класса [CarWindow](#), реализующего окно с информацией о доступных поездках.

7.3 carwindow.h

См. документацию.

```

00001
00006 #ifndef CARWINDOW_H
00007 #define CARWINDOW_H
00008
00009 #include <QDialog>
00010 #include <QVariantMap>
00011
00012 namespace Ui {
00013 class CarWindow;
00014 }
00015
00020 class CarWindow : public QDialog
00021 {
00022     Q_OBJECT
00023
00024 public:
00029     explicit CarWindow(QWidget *parent = nullptr);
00030
00034     ~CarWindow();
00035
00040     void setTripId(int tripId);
00041
00042 signals:
00046     void returnToPreviousWindow();
00047
00051     void goToFinishWindow();
00052
00053 public slots:
00057     void on_toolButton_then0_clicked();
00058
00062     void on_toolButton_then1_clicked();
00063
00068     void displayTripInfo(const QVariantMap& tripInfo);
00069
00070 private:
00071     Ui::CarWindow *ui;
00072     int currentTripId;
00073     QVariantMap currentTrip;
00074 };
00075
00076 #endif // CARWINDOW_H

```

7.4 Файл client/companionwindow.cpp

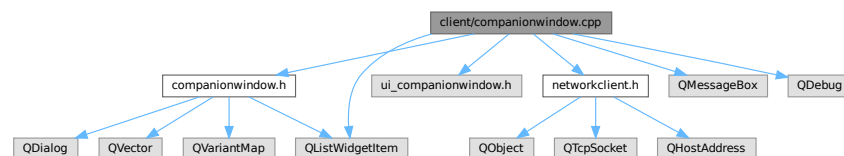
Файл реализации класса [CompanionWindow](#).

```

#include "companionwindow.h"
#include "ui_companionwindow.h"
#include "networkclient.h"
#include <QMessageBox>
#include <QDebug>
#include <QListWidgetItem>

```

Граф включаемых заголовочных файлов для companionwindow.cpp:



7.4.1 Подробное описание

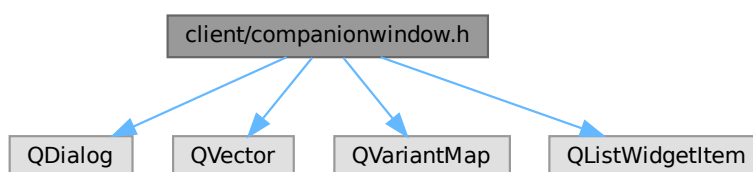
Файл реализации класса [CompanionWindow](#).

7.5 Файл client/companionwindow.h

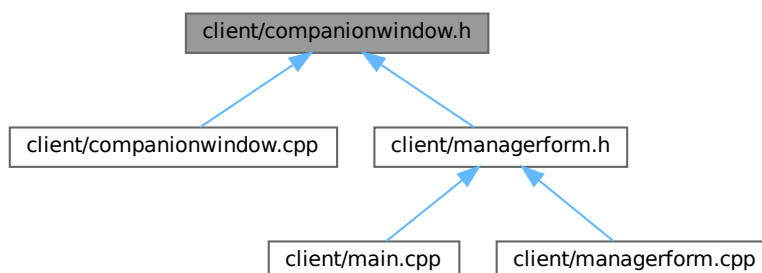
Заголовочный файл класса [CompanionWindow](#), реализующего окно для выбора и бронирования поездов пассажиром.

```
#include <QDialog>
#include <QVector>
#include <QVariantMap>
#include <QListWidgetItem>
```

Граф включаемых заголовочных файлов для companionwindow.h:



Граф файлов, в которые включается этот файл:



Классы

- class [CompanionWindow](#)

Класс [CompanionWindow](#) представляет окно для выбора и бронирования поездов пассажиром. Окно позволяет искать доступные поездки, выбирать их и бронировать.

Пространства имен

- namespace [Ui](#)

7.5.1 Подробное описание

Заголовочный файл класса [CompanionWindow](#), реализующего окно для выбора и бронирования поездов пассажиром.

7.6 companionwindow.h

[См. документацию.](#)

```

00001
00006 #ifndef COMPANIONWINDOW_H
00007 #define COMPANIONWINDOW_H
00008
00009 #include <QDialog>
00010 #include <QVector>
00011 #include <QVariantMap>
00012 #include <QListWidgetItem>
00013
00014 namespace Ui {
00015 class CompanionWindow;
00016 }
00017
00022 class CompanionWindow : public QDialog
00023 {
00024     Q_OBJECT
00025
00026 public:
00031     explicit CompanionWindow(QWidget *parent = nullptr);
00032
00036     ~CompanionWindow();
00037
00042     QVector<QVariantMap> getAvailableTrips() const;
00043
00044 signals:
00048     void returnToPreviousWindow();
00049
00055     void goToCarWindow(int tripId, QVariantMap tripInfo);
00056
00060     void tripNotFound();
00061
00065     void goToDriverCompanionWindow();
00066
00067 public slots:
00071     void on_toolButton_then0_clicked();
00072
00076     void on_toolButton_then1_clicked();
00077
00082     void handleFindTripResponse(const QString& message);
00083
00088     void handleBookTripResponse(const QString& message);
00089
00094     void on_listWidget_info_itemClicked(QListWidgetItem *item);
00095
00096 private:
00097     Ui::CompanionWindow *ui;
00098     QVector<QVariantMap> availableTrips;
00099     int selectedTripId = -1;
00100     bool isSearchPerformed = false;
00101 };
00102
00103 #endif // COMPANIONWINDOW_H

```

7.7 Файл client/drivercompanionwindow.cpp

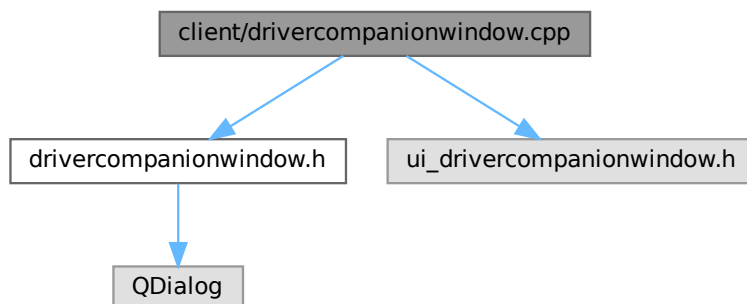
Файл реализации класса [DriverCompanionWindow](#).

```

#include "drivercompanionwindow.h"
#include "ui_drivercompanionwindow.h"

```

Граф включаемых заголовочных файлов для `drivercompanionwindow.cpp`:



7.7.1 Подробное описание

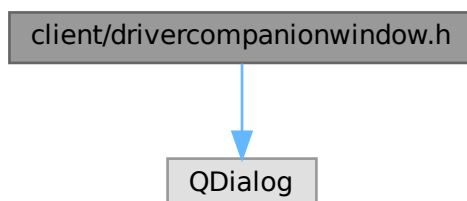
Файл реализации класса [DriverCompanionWindow](#).

7.8 Файл `client/drivercompanionwindow.h`

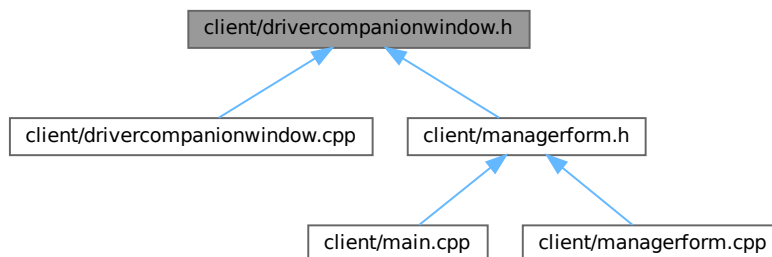
Заголовочный файл класса [DriverCompanionWindow](#), реализующего окно выбора роли (пассажир, водитель).

```
#include <QDialog>
```

Граф включаемых заголовочных файлов для `drivercompanionwindow.h`:



Граф файлов, в которые включается этот файл:



Классы

- class [DriverCompanionWindow](#)

Класс [DriverCompanionWindow](#) представляет окно для выбора роли водителя, пассажира, или оставления отзыва. Окно предоставляет возможность возврата на предыдущий экран или перехода к различным экранам.

Пространства имен

- namespace [Ui](#)

7.8.1 Подробное описание

Заголовочный файл класса [DriverCompanionWindow](#), реализующего окно выбора роли (пассажир, водитель).

7.9 drivercompanionwindow.h

[См. документацию.](#)

```

00001
00006 #ifndef DRIVERCOMPANIONWINDOW_H
00007 #define DRIVERCOMPANIONWINDOW_H
00008
00009 #include <QDialog>
00010
00011 namespace Ui {
00012 class DriverCompanionWindow;
00013 }
00014
00015 class DriverCompanionWindow : public QDialog
00016 {
00017     Q_OBJECT
00018
00019 public:
00020     explicit DriverCompanionWindow(QWidget *parent = nullptr);
00021     ~DriverCompanionWindow();
00022
00023 signals:
00024     void returnToPreviousWindow();
00025     void goToCompanionWindow();
00026
00027 private:
00028
00029
00030
00031
00032
00033
00034
00035
00036
00037
00038
00039
00040
00041
00042
00043
00044
00045
  
```

```

00049 void goToDriverWindow();
00050
00054 void goToFeedbackWindow();
00055
00056 private slots:
00060 void on_toolButton_then0_clicked();
00061
00065 void on_pushButton_companion_clicked();
00066
00070 void on_pushButton_driver_clicked();
00071
00075 void on_pushButton_feedback_clicked();
00076
00077 private:
00078     Ui::DriverCompanionWindow *ui;
00079 };
00080
00081 #endif // DRIVERCOMPANIONWINDOW_H

```

7.10 Файл client/driverwindow.cpp

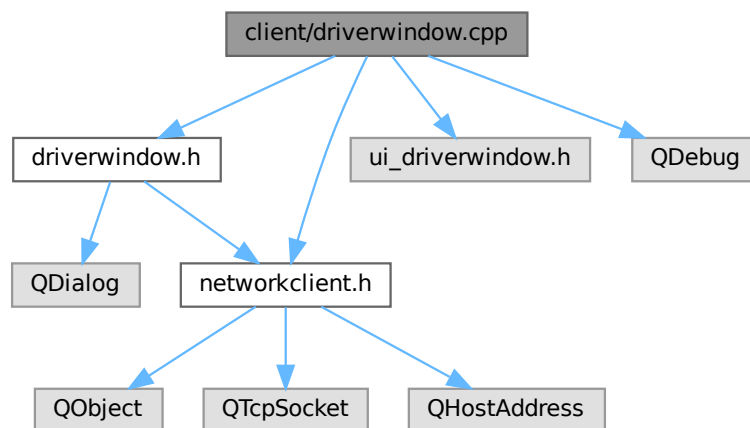
Файл реализации класса [DriverWindow](#).

```

#include "driverwindow.h"
#include "ui_driverwindow.h"
#include "networkclient.h"
#include <QDebug>

```

Граф включаемых заголовочных файлов для driverwindow.cpp:



7.10.1 Подробное описание

Файл реализации класса [DriverWindow](#).

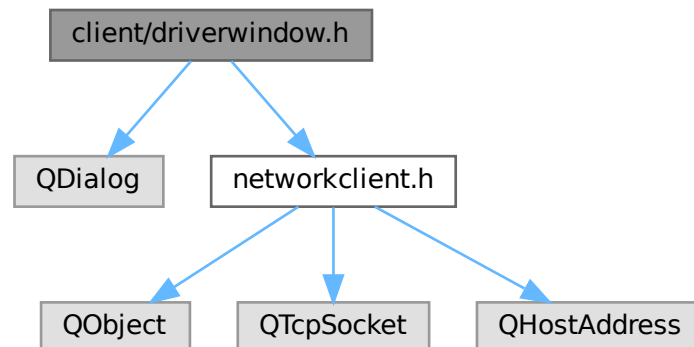
7.11 Файл client/driverwindow.h

Заголовочный файл класса `DriverWindow`, реализующего окно для водителя.

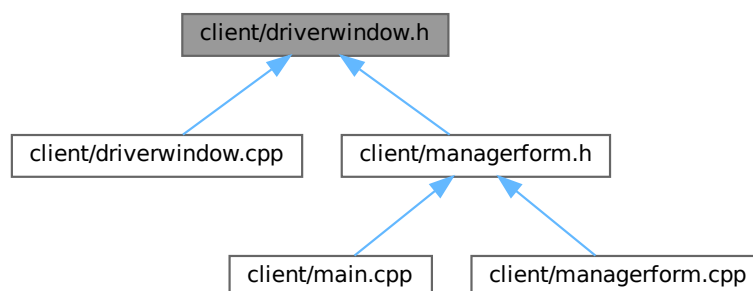
```
#include <QDialog>
```

```
#include "networkclient.h"
```

Граф включаемых заголовочных файлов для driverwindow.h:



Граф файлов, в которые включается этот файл:



Классы

- class `DriverWindow`

Класс для представления окна водителя.

Пространства имен

- namespace `Ui`

7.11.1 Подробное описание

Заголовочный файл класса [DriverWindow](#), реализующего окно для водителя.

7.12 driverwindow.h

[См. документацию.](#)

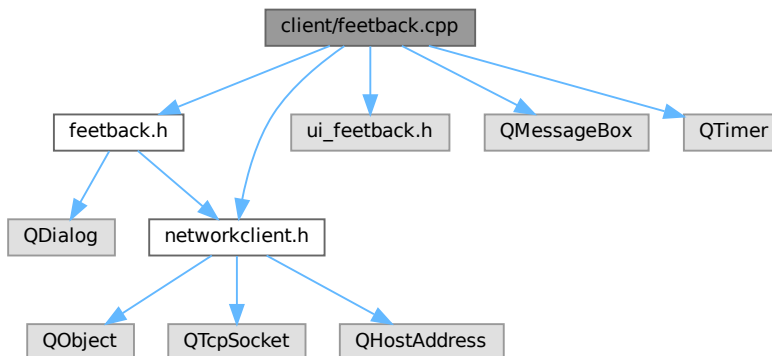
```
00001
00007 #ifndef DRIVERWINDOW_H
00008 #define DRIVERWINDOW_H
00009
00010 #include <QDialog>
00011 #include "networkclient.h" // Добавляем заголовочный файл NetworkClient
00012
00013 namespace Ui {
00014 class DriverWindow;
00015 }
00016
00022 class DriverWindow : public QDialog
00023 {
00024     Q_OBJECT
00025
00026 public:
00034     explicit DriverWindow(QWidget *parent = nullptr);
00035
00041     ~DriverWindow();
00042
00043 signals:
00047     void returnToPreviousWindow();
00048
00052     void goToFinishWindow();
00053
00054 private slots:
00060     void on_toolButton_then0_clicked();
00061
00067     void on_toolButton_then1_clicked();
00068
00069 private:
00070     Ui::DriverWindow *ui;
00071 };
00072
00073 #endif // DRIVERWINDOW_H
```

7.13 Файл client/feedback.cpp

Файл реализации класса [Trips](#).

```
#include "feedback.h"
#include "ui_feedback.h"
#include "networkclient.h"
#include <QMessageBox>
#include <QTimer>
```


Граф включаемых заголовочных файлов для feedback.cpp:



7.13.1 Подробное описание

Файл реализации класса [Ttips](#).

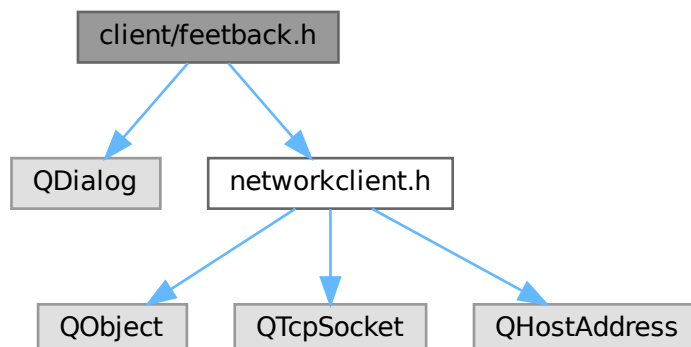
7.14 Файл client/feedback.h

Заголовочный файл класса [Ttips](#), реализующего возможность оставления отзывов и рейтинга поездок.

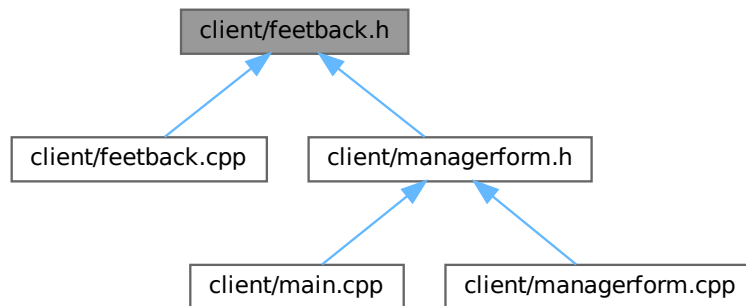
```
#include <QDialog>
```

```
#include "networkclient.h"
```

Граф включаемых заголовочных файлов для feedback.h:



Граф файлов, в которые включается этот файл:



Классы

- class [Tpips](#)
Класс окна обратной связи.

Пространства имен

- namespace [Ui](#)

7.14.1 Подробное описание

Заголовочный файл класса [Tpips](#), реализующего возможность оставления отзывов и рейтинга поездов.

7.15 feedback.h

[См. документацию.](#)

```

00001
00006 #ifndef FEETBACK_H
00007 #define FEETBACK_H
00008
00009 #include <QDialog>
00010 #include "networkclient.h" // Добавляем заголовочный файл NetworkClient
00011
00012 namespace Ui {
00013     class Tpips;
00014 }
00015
00022 class Tpips : public QDialog
00023 {
00024     Q_OBJECT
00025
00026 public:
00033     explicit Tpips(QWidget *parent = nullptr);
00034
00040     ~Tpips();
00041
00042 signals:
00046     void finished();
00047
  
```

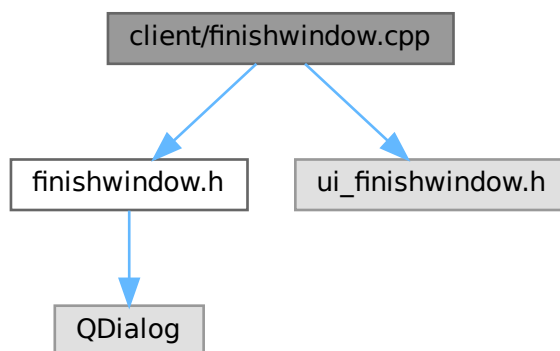
```
00051 void goToDriverCompanionWindow();
00052
00053 private slots:
00059 void on_toolButton_0_clicked();
00060
00066 void on_toolButton_1_clicked();
00067
00068 private:
00069     Ui::Trips *ui;
00070 };
00071
00072 #endif // FEETBACK_H
```

7.16 Файл client/finishwindow.cpp

Файл реализации класса [FinishWindow](#).

```
#include "finishwindow.h"
#include "ui_finishwindow.h"
```

Граф включаемых заголовочных файлов для finishwindow.cpp:



7.16.1 Подробное описание

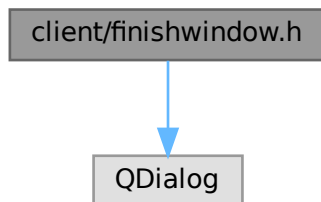
Файл реализации класса [FinishWindow](#).

7.17 Файл client/finishwindow.h

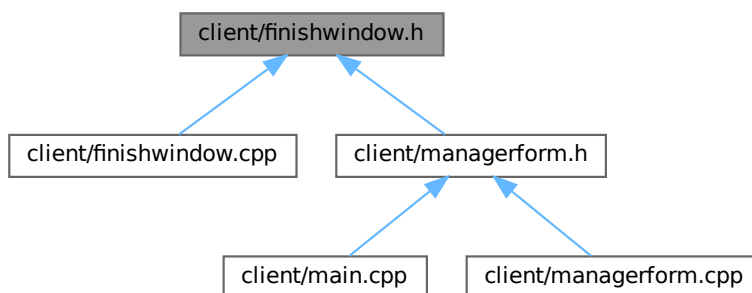
Заголовочный файл класса [FinishWindow](#), реализующего окно завершения.

```
#include <QDialog>
```

Граф включаемых заголовочных файлов для finishwindow.h:



Граф файлов, в которые включается этот файл:



Классы

- class [FinishWindow](#)
Класс окна завершения.

Пространства имен

- namespace [Ui](#)

7.17.1 Подробное описание

Заголовочный файл класса [FinishWindow](#), реализующего окно завершения.

7.18 finishwindow.h

См. документацию.

```

00001
00006 #ifndef FINISHWINDOW_H
00007 #define FINISHWINDOW_H
00008
00009 #include <QDialog>
00010
00011 namespace Ui {
00012     class FinishWindow;
00013 }
00014
00021 class FinishWindow : public QDialog
00022 {
00023     Q_OBJECT
00024
00025 signals:
00029     void returnToMainWindow();
00030
00031 public:
00038     explicit FinishWindow(QWidget *parent = nullptr);
00039
00045     ~FinishWindow();
00046
00047 private slots:
00051     void on_pushButton_home_clicked();
00052
00053 private:
00054     Ui::FinishWindow *ui;
00055 };
00056
00057 #endif // FINISHWINDOW_H

```

7.19 Файл client/loginwindow.cpp

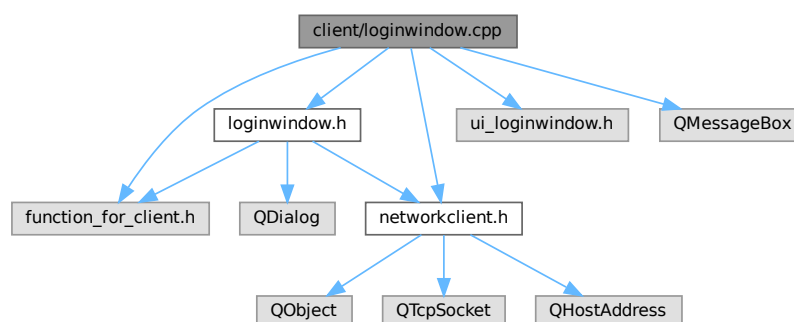
Файл реализации класса [LoginWindow](#).

```

#include "loginwindow.h"
#include "function_for_client.h"
#include "ui_loginwindow.h"
#include "networkclient.h"
#include <QMessageBox>

```

Граф включаемых заголовочных файлов для loginwindow.cpp:



7.19.1 Подробное описание

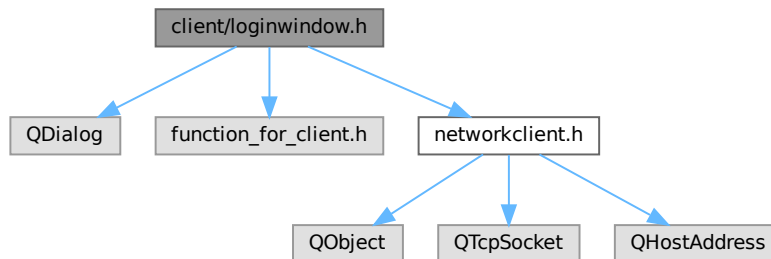
Файл реализации класса [LoginWindow](#).

7.20 Файл client/loginwindow.h

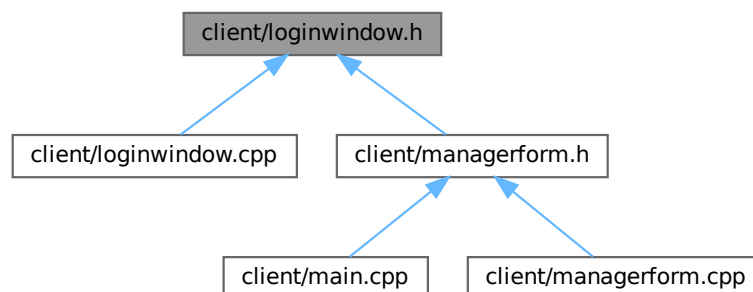
Заголовочный файл класса [LoginWindow](#), реализующего окно авторизации.

```
#include <QDialog>
#include "function_for_client.h"
#include "networkclient.h"
```

Граф включаемых заголовочных файлов для loginwindow.h:



Граф файлов, в которые включается этот файл:



Классы

- class [LoginWindow](#)
Класс окна авторизации.

Пространства имен

- namespace [Ui](#)

7.20.1 Подробное описание

Заголовочный файл класса [LoginWindow](#), реализующего окно авторизации.

7.21 loginwindow.h

См. документацию.

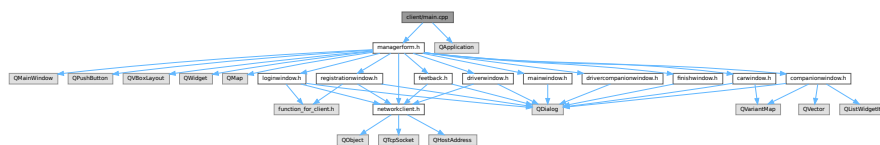
```
00001
00006 #ifndef LOGINWINDOW_H
00007 #define LOGINWINDOW_H
00008
00009 #include <QDialog>
00010 #include "function_for_client.h"
00011 #include "networkclient.h"
00012 namespace Ui {
00013     class LoginWindow;
00014 }
00015
00023 class LoginWindow : public QDialog
00024 {
00025     Q_OBJECT
00026
00027 public:
00034     explicit LoginWindow(QWidget *parent = nullptr);
00035
00041     ~LoginWindow();
00042
00043 signals:
00047     void returnToMainWindow();
00048
00052     void goToDriverCompanionWindow();
00053
00054 private slots:
00058     void on_toolButton_then0_clicked();
00059
00063     void on_toolButton_then1_clicked();
00064
00070     void onAuthSuccess();
00071
00077     void onAuthFailed();
00078
00084     void onError(const QString& message);
00085
00086 private:
00087     Ui::LoginWindow *ui;
00092     void clear();
00093 };
00094
00095 #endif // LOGINWINDOW_H
```

7.22 Файл client/main.cpp

Главный файл приложения.

```
#include "managerform.h"
#include <QApplication>
```

Граф включаемых заголовочных файлов для `main.cpp`:



Функции

- `int main (int argc, char *argv[])`

7.22.1 Подробное описание

Главный файл приложения.

Этот файл инициализирует приложение, создаёт главное окно и запускает основной цикл обработки событий.

7.22.2 Функции

7.22.2.1 main()

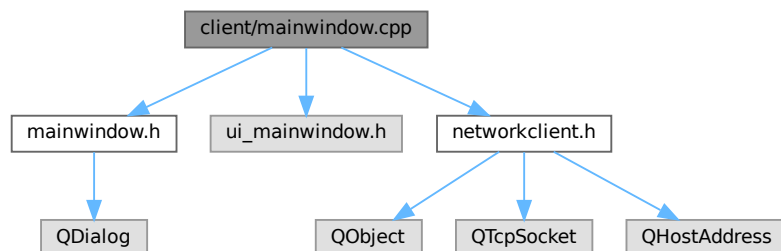
```
int main (
    int argc,
    char * argv[] )
```

7.23 Файл client/mainwindow.cpp

Файл реализации класса [MainWindow](#).

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include "networkclient.h"
```

Граф включаемых заголовочных файлов для mainwindow.cpp:



7.23.1 Подробное описание

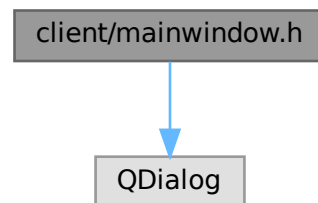
Файл реализации класса [MainWindow](#).

7.24 Файл client/mainwindow.h

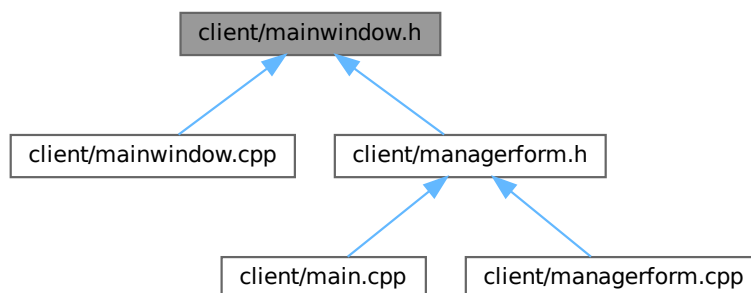
Заголовочный файл класса [MainWindow](#), реализующего главное окно приложения.

```
#include <QDialog>
```

Граф включаемых заголовочных файлов для mainwindow.h:



Граф файлов, в которые включается этот файл:



Классы

- class [MainWindow](#)
Класс главного окна приложения.

Пространства имен

- namespace [Ui](#)

7.24.1 Подробное описание

Заголовочный файл класса [MainWindow](#), реализующего главное окно приложения.

7.25/mainwindow.h

См. документацию.

```

00001
00006 #ifndef MAINWINDOW_H
00007 #define MAINWINDOW_H
00008
00009 #include <QDialog>
00010
00011 namespace Ui {
00012     class MainWindow;
00013 }
00014
00023 class MainWindow : public QDialog
00024 {
00025     Q_OBJECT
00026
00027 public:
00034     explicit MainWindow(QWidget *parent = nullptr);
00035
00041     ~MainWindow();
00042
00043 signals:
00047     void loginButtonClicked();
00048
00052     void registrationButtonClicked();
00053
00054 private slots:
00060     void on_pushButton_auth_clicked();
00061
00067     void on_pushButton_reg_clicked();
00068
00069 private:
00070     Ui::MainWindow *ui;
00071 };
00072
00073 #endif // MAINWINDOW_H

```

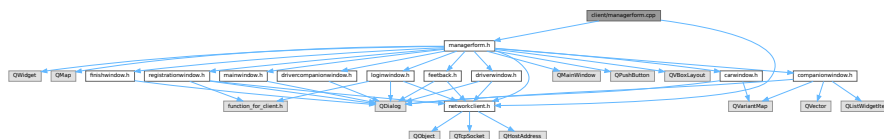
7.26 Файл client/managerform.cpp

Файл реализации класса [ManagerForm](#).

```
#include "managerform.h"
```

```
#include "networkclient.h"
```

Граф включаемых заголовочных файлов для managerform.cpp:



7.26.1 Подробное описание

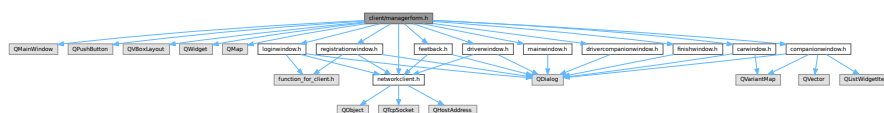
Файл реализации класса [ManagerForm](#).

7.27 Файл client/managerform.h

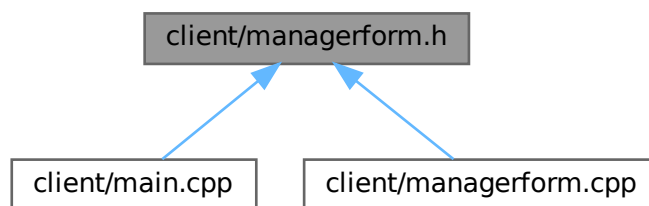
Заголовочный файл класса [ManagerForm](#) для управления окнами приложения.

```
#include <QMainWindow>
#include <QPushButton>
#include <QVBoxLayout>
#include <QWidget>
#include <QMap>
#include "mainwindow.h"
#include "loginwindow.h"
#include "registrationwindow.h"
#include "drivercompanionwindow.h"
#include "companionwindow.h"
#include "driverwindow.h"
#include "carwindow.h"
#include "finishwindow.h"
#include "networkclient.h"
#include "feedback.h"
```

Граф включаемых заголовочных файлов для managerform.h:



Граф файлов, в которые включается этот файл:



7.27.1 Подробное описание

Заголовочный файл класса [ManagerForm](#) для управления окнами приложения.

7.28 managerform.h

См. документацию.
00001

```

00006 #ifndef MANAGERFORM_H
00007 #define MANAGERFORM_H
00008
00009 #include <QMainWindow>
00010 #include <QPushButton>
00011 #include <QVBoxLayout>
00012 #include <QWidget>
00013 #include <QMap>
00014 #include "mainwindow.h"
00015 #include "loginwindow.h"
00016 #include "registrationwindow.h"
00017 #include "drivercompanionwindow.h"
00018 #include "companionwindow.h"
00019 #include "driverwindow.h"
00020 #include "carwindow.h"
00021 #include "finishwindow.h"
00022 #include "networkclient.h"
00023 #include "feedback.h"
00024
00032 class ManagerForm : public QMainWindow
00033 {
00034     Q_OBJECT
00035
00036 public:
00041     explicit ManagerForm(QWidget *parent = nullptr);
00042
00046     ~ManagerForm();
00047
00048 private slots:
00052     void showLoginWindow();
00053
00057     void showRegistrationWindow();
00058
00062     void showDriverCompanionWindow();
00063
00067     void showCompanionWindow();
00068
00072     void showDriverWindow();
00073
00079     void showCarWindow(int tripId, QMap tripInfo);
00080
00084     void handleReturnToPrevious();
00085
00089     void showFinishWindow();
00090
00094     void showMainWindowFromFinish();
00095
00100     void onConnectionStatusChanged(bool connected);
00101
00105     void showFeedbackWindow();
00106
00110     void showDriverCompanionWindowFromCompanion();
00111
00112 private:
00113     MainWindow *Main_Window;
00114     LoginWindow *Login_Window;
00115     RegistrationWindow *Reg_Window;
00116     DriverCompanionWindow *Drive_Comp_Window;
00117     CompanionWindow *Comp

```

7.29 Файл client/networkclient.cpp

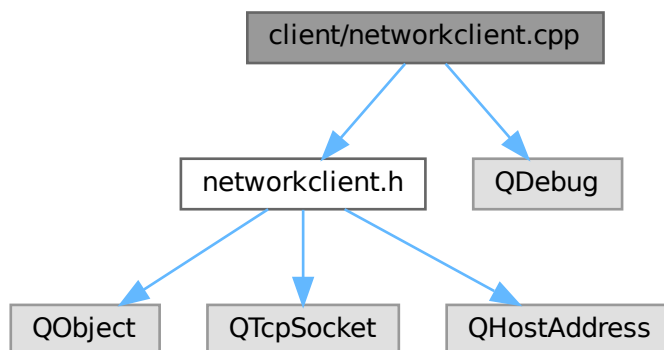
Файл реализации класса [RegistrationWindow](#).

```

#include "networkclient.h"
#include <QDebug>

```

Граф включаемых заголовочных файлов для networkclient.cpp:



7.29.1 Подробное описание

Файл реализации класса [RegistrationWindow](#).

7.30 Файл client/networkclient.h

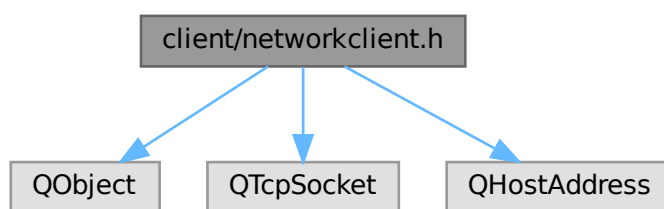
Заголовочный файл класса [RegistrationWindow](#) для работы с сетевым клиентом.

```

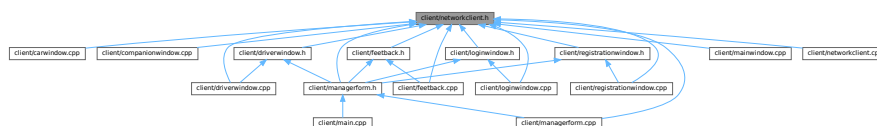
#include <QObject>
#include <QTcpSocket>
#include <QHostAddress>

```

Граф включаемых заголовочных файлов для networkclient.h:



Граф файлов, в которые включается этот файл:



Классы

- class [NetworkClient](#)

Класс для работы с сетевым клиентом.

7.30.1 Подробное описание

Заголовочный файл класса [RegistrationWindow](#) для работы с сетевым клиентом.

7.31 networkclient.h

[См. документацию.](#)

```

00001
00005 #ifndef NETWORKCLIENT_H
00006 #define NETWORKCLIENT_H
00007
00008 #include <QObject>
00009 #include <QTcpSocket>
00010 #include <QHostAddress>
00011
00020 class NetworkClient : public QObject
00021 {
00022     Q_OBJECT
00023
00024 public:
00029     static NetworkClient& getInstance()
00030     {
00031         static NetworkClient instance;
00032         return instance;
00033     }
00034
00035     // Запрещаем создание копий и присваивание, чтобы гарантировать единственность экземпляра
00036     NetworkClient(NetworkClient const&) = delete;
00037     void operator=(NetworkClient const&) = delete;
00038
00044     void connectToServer(const QString& host, quint16 port);
00049     void sendMessage(const QString& message);
00050
00055     QString getLogin() const { return login; }
00056
00057     signals:
00062     void connectionStatusChanged(bool connected);
00063
00068     void error(const QString& message);
00069
00073     void authSuccess();
00074
00078     void authFailed();
00079
00083     void regSuccess();
00084
00088     void regFailed();
00089
00094     void readyRead(const QString& message);
00095
00100     void bookTripResponse(const QString& response);
00101
00102 public slots:
00106     void connectedToServer();
00107
00111     void disconnectedFromServer();
00112
00117     void socketError(QAbstractSocket::SocketError socketError);
00118
00122     void _readyRead();
00123
00124 private:
00128     NetworkClient();
00129
00133     ~NetworkClient() override;
00134
00135     QTcpSocket* socket;
00136     QString serverHost;
00137     quint16 serverPort;
00138     bool isConnected;
00139     QString login;
00140 };
00141
00142 #endif // NETWORKCLIENT_H

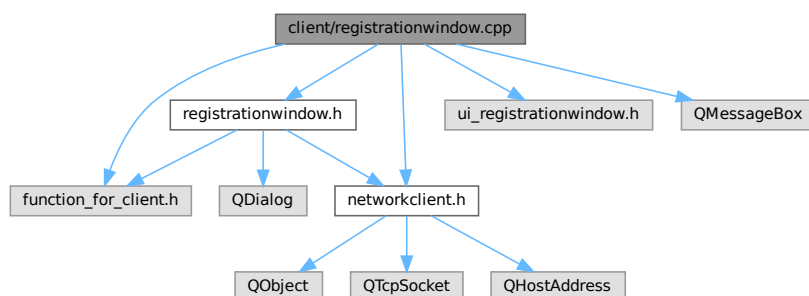
```

7.32 Файл client/registrationwindow.cpp

Файл реализации класса [RegistrationWindow](#).

```
#include "registrationwindow.h"
#include "ui_registrationwindow.h"
#include "function_for_client.h"
#include "networkclient.h"
#include <QMessageBox>
```

Граф включаемых заголовочных файлов для registrationwindow.cpp:



7.32.1 Подробное описание

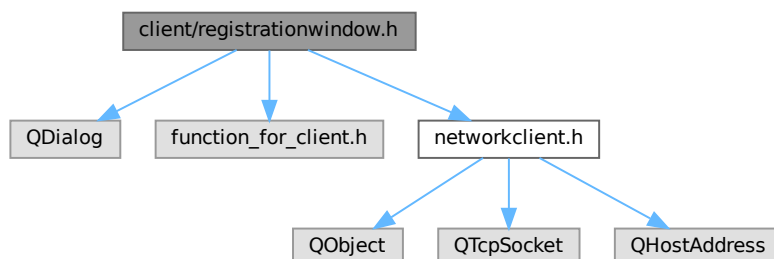
Файл реализации класса [RegistrationWindow](#).

7.33 Файл client/registrationwindow.h

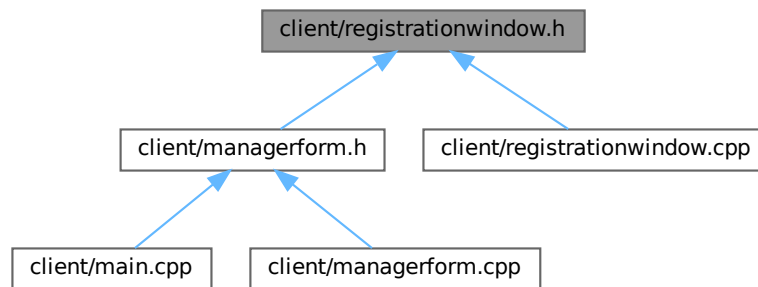
Заголовочный файл класса [RegistrationWindow](#) для реализации окна регистрации.

```
#include <QDialog>
#include "function_for_client.h"
#include "networkclient.h"
```

Граф включаемых заголовочных файлов для registrationwindow.h:



Граф файлов, в которые включается этот файл:



Классы

- class [RegistrationWindow](#)
Класс окна регистрации пользователя.

Пространства имен

- namespace [Ui](#)

7.33.1 Подробное описание

Заголовочный файл класса [RegistrationWindow](#) для реализации окна регистрации.

7.34 registrationwindow.h

[См. документацию.](#)

```

00001
00005 #ifndef REGISTRATIONWINDOW_H
00006 #define REGISTRATIONWINDOW_H
00007
00008 #include <QDialog>
00009 #include "function_for_client.h"
00010 #include "networkclient.h" // Добавляем заголовочный файл
00011
00012 namespace Ui {
00013 class RegistrationWindow;
00014 }
00015
00023 class RegistrationWindow : public QDialog
00024 {
00025     Q_OBJECT
00026
00027 public:
00032     explicit RegistrationWindow(QWidget *parent = nullptr);
00036     ~RegistrationWindow();
00037
00038 signals:
00042     void returnToMainWindow();
00046     void goToDriverCompanionWindow();
00047
00048 private slots:
00052     void on_toolButton_then0_clicked();
  
```



```
00056     void on_toolButton_then1_clicked();
00060     void onRegSuccess();
00064     void onRegFailed();
00069     void onError(const QString& message);
00070
00071 private:
00072     Ui::RegistrationWindow *ui;
00076     void clear();
00077 };
00078
00079 #endif // REGISTRATIONWINDOW_H
```


Предметный указатель

- _readyRead
 - NetworkClient, 31
- ~CarWindow
 - CarWindow, 12
- ~CompanionWindow
 - CompanionWindow, 16
- ~DriverCompanionWindow
 - DriverCompanionWindow, 19
- ~DriverWindow
 - DriverWindow, 22
- ~FinishWindow
 - FinishWindow, 24
- ~LoginWindow
 - LoginWindow, 25
- ~MainWindow
 - MainWindow, 27
- ~RegistrationWindow
 - RegistrationWindow, 35
- ~Tpips
 - Tpips, 37
- authFailed
 - NetworkClient, 31
- authSuccess
 - NetworkClient, 31
- bookTripResponse
 - NetworkClient, 31
- CarWindow, 11
 - ~CarWindow, 12
 - CarWindow, 12
 - displayTripInfo, 13
 - goToFinishWindow, 13
 - on_toolButton_then0_clicked, 13
 - on_toolButton_then1_clicked, 13
 - returnToPreviousWindow, 13
 - setTripId, 13
- client/carwindow.cpp, 39
- client/carwindow.h, 39, 41
- client/companionwindow.cpp, 41
- client/companionwindow.h, 42, 43
- client/drivercompanionwindow.cpp, 43
- client/drivercompanionwindow.h, 44, 45
- client/driverwindow.cpp, 46
- client/driverwindow.h, 47, 48
- client/feedback.cpp, 48
- client/feedback.h, 49, 50
- client/finishwindow.cpp, 51
- client/finishwindow.h, 51, 53
- client/loginwindow.cpp, 53
- client/loginwindow.h, 54, 55
- client/main.cpp, 55
- client/mainwindow.cpp, 56
- client/mainwindow.h, 57, 58
- client/managerform.cpp, 58
- client/managerform.h, 59
- client/networkclient.cpp, 60
- client/networkclient.h, 61, 62
- client/registrationwindow.cpp, 63
- client/registrationwindow.h, 63, 64
- CompanionWindow, 14
 - ~CompanionWindow, 16
 - CompanionWindow, 15
 - getAvailableTrips, 16
 - goToCarWindow, 16
 - goToDriverCompanionWindow, 16
 - handleBookTripResponse, 16
 - handleFindTripResponse, 17
 - on_listWidget_info_itemClicked, 17
 - on_toolButton_then0_clicked, 17
 - on_toolButton_then1_clicked, 17
 - returnToPreviousWindow, 17
 - tripNotFound, 17
- connectedToServer
 - NetworkClient, 31
- connectionStatusChanged
 - NetworkClient, 31
- connectToServer
 - NetworkClient, 32
- disconnectedFromServer
 - NetworkClient, 32
- displayTripInfo
 - CarWindow, 13
- DriverCompanionWindow, 18
 - ~DriverCompanionWindow, 19
 - DriverCompanionWindow, 19
 - goToCompanionWindow, 19
 - goToDriverWindow, 19
 - goToFeedbackWindow, 20
 - returnToPreviousWindow, 20
- DriverWindow, 20
 - ~DriverWindow, 22
 - DriverWindow, 21
 - goToFinishWindow, 22
 - returnToPreviousWindow, 22
- error

- NetworkClient, 32
- finished
 - Tpips, 37
- FinishWindow, 22
 - ~FinishWindow, 24
 - FinishWindow, 23
 - returnToMainWindow, 24
- getAvailableTrips
 - CompanionWindow, 16
- getInstance
 - NetworkClient, 32
- getLogin
 - NetworkClient, 32
- goToCarWindow
 - CompanionWindow, 16
- goToCompanionWindow
 - DriverCompanionWindow, 19
- goToDriverCompanionWindow
 - CompanionWindow, 16
 - LoginWindow, 25
 - RegistrationWindow, 35
 - Tpips, 37
- goToDriverWindow
 - DriverCompanionWindow, 19
- goToFeedbackWindow
 - DriverCompanionWindow, 20
- goToFinishWindow
 - CarWindow, 13
 - DriverWindow, 22
- handleBookTripResponse
 - CompanionWindow, 16
- handleFindTripResponse
 - CompanionWindow, 17
- loginButtonClicked
 - MainWindow, 28
- LoginWindow, 24
 - ~LoginWindow, 25
 - goToDriverCompanionWindow, 25
 - LoginWindow, 25
 - returnToMainWindow, 25
- main
 - main.cpp, 56
- main.cpp
 - main, 56
- MainWindow, 26
 - ~MainWindow, 27
 - loginButtonClicked, 28
 - MainWindow, 27
 - registrationButtonClicked, 28
- ManagerForm, 28
- NetworkClient, 29
 - _readyRead, 31
 - authFailed, 31
 - authSuccess, 31
 - bookTripResponse, 31
 - connectedToServer, 31
 - connectionStatusChanged, 31
 - connectToServer, 32
 - disconnectedFromServer, 32
 - error, 32
 - getInstance, 32
 - getLogin, 32
 - NetworkClient, 30
 - operator=, 32
 - readyRead, 33
 - regFailed, 33
 - regSuccess, 33
 - sendMessage, 33
 - socketError, 33
- on_listWidget_info_itemClicked
 - CompanionWindow, 17
- on_toolButton_then0_clicked
 - CarWindow, 13
 - CompanionWindow, 17
- on_toolButton_then1_clicked
 - CarWindow, 13
 - CompanionWindow, 17
- operator=
 - NetworkClient, 32
- readyRead
 - NetworkClient, 33
- regFailed
 - NetworkClient, 33
- registrationButtonClicked
 - MainWindow, 28
- RegistrationWindow, 34
 - ~RegistrationWindow, 35
 - goToDriverCompanionWindow, 35
 - RegistrationWindow, 35
 - returnToMainWindow, 35
- regSuccess
 - NetworkClient, 33
- returnToMainWindow
 - FinishWindow, 24
 - LoginWindow, 25
 - RegistrationWindow, 35
- returnToPreviousWindow
 - CarWindow, 13
 - CompanionWindow, 17
 - DriverCompanionWindow, 20
 - DriverWindow, 22
- sendMessage
 - NetworkClient, 33
- setTripId
 - CarWindow, 13
- socketError
 - NetworkClient, 33
- Tpips, 36
 - ~Tpips, 37

finished, [37](#)
goToDriverCompanionWindow, [37](#)
Trips, [37](#)
tripNotFound
CompanionWindow, [17](#)

Ui, [9](#)